

Programare Orientată Pe Obiecte

Curs 3, 4

Clase instanțiabile

Crearea și asignarea obiectelor, constructorii

Metode: vizibilitate, supraîncărcare

Metode și variabile statice

Tablouri

Octombrie 2025

Clase instanțiable

Clasă instanțiabilă

- O clasă este instanțiabilă dacă putem crea instanțe ale clasei respective
- Exemple: clasele învelitoare ale tipurilor primitive, String, Scanner, sunt toate instanțiabile, în timp ce clasa Math nu este
- Fiecare obiect este membru al unei clase: cont1 este un obiect membru al clasei ContBancar
- Avem o relație de tipul **este-o**
- La crearea unei clase trebuie:
 - Să dați un nume clasei
 - Să determinați ce date și metode vor fi parte a clasei

Modificatori de acces pentru clase

- **public:**

- Cel mai folosit; clasa este vizibilă tuturor
- Clasa poate fi extinsă sau folosită ca bază pentru alte clase

```
public class PublicClass { // Accessible from everywhere }
```

- **default:** clasa este vizibilă în interiorul pachetului;

```
class DefaultClass { // Accessible from package }
```

- **final:** folosit doar în circumstanțe speciale (clasa este complet definită și nu sunt necesare definiții de subclase)
- **abstract:** folosit doar în circumstanțe speciale (clasa este incomplet definită – conține o metodă neimplementată)
- Specificatorii de acces **private** și **protected** se pot folosi doar pentru clase interne sau pentru variabile instanță / metode !

Blocuri și vizibilitate

- Bloc: în orice clasă sau metodă, conține codul inclus între o pereche de acolade
- Porțiunea de program în care se poate referi o variabilă constituie domeniul de vizibilitate (scop) al variabilei
- O variabilă există, sau devine vizibilă la declarare
- O variabilă încetează să existe sau devine invizibilă la sfârșitul blocului în care a fost declarată
- Dacă declarați o variabilă într-o clasă și folosiți același nume pentru a declara o variabilă într-o metodă a clasei, variabila declarată în metodă are precedența sau suprascrie, prima variabilă

```
public void modificaSold(){ // in Clasa ContBancar.java
    int sold = 50;
    sold += 40;
    System.out.println("Sold: " + sold + " this.sold:" +this.sold);
}
```

Constante

În Java constantele sunt declarate cu ajutorul cuvântului cheie **final**.

- **final** se poate folosi și la variabile non-stactice, inclusiv la variabile instanța sau la parametrii metodelor.
- Semnificatia e aceeași: valoarea variabilei nu poate fi schimbată
- **final** se mai folosește ca să se oprească supraincarcarea sau moștenirea.

```
public class Color {  
    public static final Color BLACK = new Color(0,0,0);  
    public static final Color BLUE = new Color(0,0,255);  
    public static final Color GREEN= new Color(0,255,0);  
    ...  
}
```

Crearea și Asignarea Obiectelor

Crearea Obiectelor (1)

- Java are trei mecanisme dedicate asigurării inițializării corespunzătoare a obiectelor:
 - inițializatori de instanță (numiți și blocuri de inițializare de instanță)
 - inițializatori de variabile instanță
 - **constructori**
- Toate cele trei mecanisme presupun cod executat automat la crearea unui obiect
- La alocarea memoriei pentru un nou obiect folosind operatorul `new` sau metoda `newInstance()` a clasei `Class`, JVM asigură executarea codului de inițializare înainte de folosirea zonei alocate

Crearea Obiectelor (2)

Exemplu

La invocarea operatorului `new`:

- Se alocă memorie (se rezervă spațiu pentru obiect). Variabilele instanță sunt inițializate la valorile lor implicite
- Se execută inițializarea explicită. Variabilele inițializate la declararea atributelor primesc valorile declarate

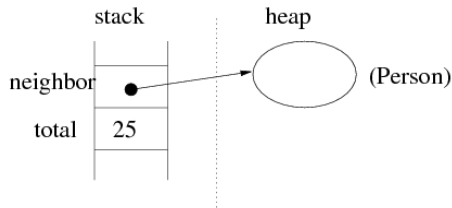
```
class Exemplu{  
    int nr = 1;  
    public Exemplu () {  
        nr = 20;  
    }  
    public static void main(){  
        Exemplu e = new Exemplu();  
        System.out.print( "Nr:" + e.nr);  
    }  
}
```

- Se execută un constructor. Valorile variabilelor pot fi schimbate de constructor
- Se atribuie variabilei o referință la obiect

Crearea Obiectelor (3)

La apelul unui constructor:

- Se alocă spațiu pe heap pentru obiect
- Fiecare obiect primește spațiu propriu (propria copie a variabilelor instanță)
- Starea obiectului este inițializată potrivit codului (definit de programator) clasei
- Declararea unei variabile ca fiind de un tip de obiect produce o referință la obiect și nu obiectul în sine
- Pentru a obține obiectul în sine folosiți **new** și un **constructor** pentru clasă
- Se poate combina declararea și inițializarea:
Person neighbor = new Person();
- La fel ca și pentru tipurile primitive:
int total = 25;



Inițializarea Câmpurilor (1)

1. Atribuire simplă: se atribuie o valoare inițială unui câmp la declararea sa

```
public static int capacity = 10; //initializat la 10
private boolean full = false; //initializat la false
```

2. Blocuri de inițializare statice: Bloc normal de cod între acolade, { } și precedat de cuvântul cheie **static**.

```
static { // codul necesar initializarii se scrie aici }
```

- Folosit la inițializarea variabilelor la nivel de clasă
- Blocurile de inițializare statice sunt executate în ordinea în care apar în codul sursă

3. Metodă statică privată: alternativă la blocurile statice

```
class Oricare {
    public static varType myVar = initializeClassVariable();
    private static varType initializeClassVariable() {
        //codul de initializare se pune aici
    }
}
```

Inițializarea Câmpurilor (2)

Inițializarea membrilor instanțelor:

- Asemănătoare blocurilor statice, dar nu static. Compilatorul Java copiază blocurile de inițializare în fiecare constructor:

Exemplu

```
{  
    // codul pentru initializare, aici  
}
```

SAU se folosește o **metodă final** pentru inițializarea unei variabile instanță (alternativă la inițializarea membrilor instanțelor)

```
class Oricare {  
    private varType myVar = initializeInstanceVariable();  
    protected final varType initializeInstanceVariable() {  
        //cod de initializare  
    }  
}
```

- Folositoare mai ales dacă subclasele doresc să refolosească metoda de inițializare
- Metoda este final deoarece apelul metodelor non-final la inițializarea instanțelor pot cauza probleme

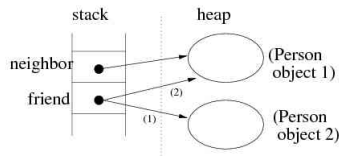
Folosirea constantelor și metodelor importate automat, de bibliotecă

- Creatorii limbajului Java au creat cca. 500 clase
- Exemple: System, Scanner, Character, Boolean, Byte, Short, Integer, Long, Float și Double sunt clase
- Aceste clase sunt stocate în pachete (biblioteci) de clase
- Un pachet este un instrument de grupare convenabilă a claselor cu funcționalitate înrudită
- `java.lang` - pachetul este importat implicit în fiecare program Java care conține clase fundamentale (de bază) - d.e. System, Character, Boolean, Byte, Short, Integer, Long, Float, Double, String
- Pentru a folosi oricare dintre clasele de bibliotecă (altele decât `java.lang`):
 - Folosiți întreaga cale împreună cu numele clasei: `area = Math.PI * raza * raza;`
 - SAU Importați clasa
 - SAU Importați pachetul care conține clasa pe care o folosiți: `import java.util.*;`

Asignarea obiectelor

```
Person neighbor = new Person(); // creeaza object1
Person friend = new Person(); // creeaza object2
friend = neighbor;
```

La sfârșit atât friend cât și neighbor se referă la object1 (ele sunt alias unul pentru celălalt) și nimic nu se mai referă la object2 (acesta este inaccesibil)



Java Garbage Collector: Java va colecta automat reziduul (zona de memorie nefolosită) object2

- De obicei metodele, variabilele și obiectele din programele Java sunt alocate în una din cele două locuri de memorie: stack sau heap
- Regula de bază este:
 - Variabilele instanță și obiectele se alocă pe heap
 - Variabilele locale se alocă pe stack

Life on the garbage-collectible heap

```
class Book {...}
```

```
Book b = new Book();  
Book c = new Book();
```

Cele două obiecte b și c "trăiesc" pe heap.

Referinte = 2; Obiecte = 2;

```
Book b = new Book();  
Book c = new Book();  
Book d = c;
```

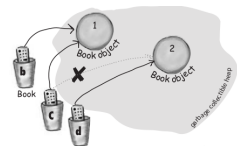
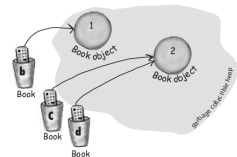
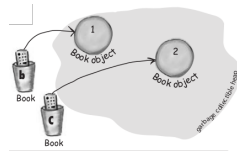
Obiectele c și d referă același obiect.

Referinte = 3; Obiecte = 2;

```
Book b = new Book();  
Book c = new Book();  
Book d = c;  
c = b;
```

Obiectele c și b referă același obiect.

Referinte = 3; Obiecte = 2;



Life and death on the heap

```
class Book {...}
```

```
Book b = new Book();
```

```
Book c = new Book();
```

Cele două obiecte b și c "trăiesc" pe heap.

Referințe = 2; Obiecte = 2;

```
Book b = new Book();
```

```
Book c = new Book();
```

```
b = c;
```

Obiectele c și b referă același obiect.

Referințe active = 2; Obiecte abandonate = 1; Obiecte active
(care se pot referi) = 1;

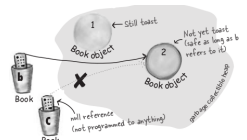
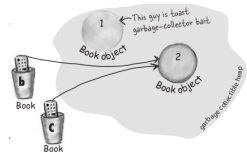
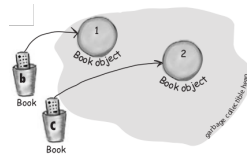
```
Book b = new Book();
```

```
Book c = new Book();
```

```
b = c;
```

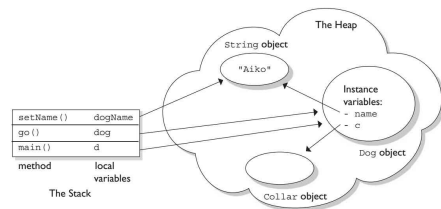
```
c = null;
```

Referința c pointeaza spre null.



Exemplu concret de cum se alocă variabilele dintr-un program Java:

```
1 class Collar { }
2 class Dog {
3     Collar c; // variabila instanta
4     String name; // variabila instanta
5     public static void main(String[] arg){
6         Dog d; //variabila locala
7         d = new Dog();
8         d.go(d);
9     }
10    void go(Dog dog) {
11        // variabila locala: dog
12        c = new Collar();
13        dog.setName("Aiko");
14    }
15    void setName(String dogName) {
16        // variabila locala: dogName
17        name = dogName;
18        // alte lucruri
19    }
20 }
```



Exemplu concret de cum se alocă variabilele dintr-un program Java:

```
1 class Collar { }
2 class Dog {
3     Collar c; // variabila instantata
4     String name; // variabila instantata
5     public static void main(String[] arg){
6         Dog d; //variabila locala
7         d = new Dog();
8         d.go(d);
9     }
10    void go(Dog dog) {
11        // variabila locala: dog
12        c = new Collar();
13        dog.setName("Aiko");
14    }
15    void setName(String dogName) {
16        // variabila locala: dogName
17        name = dogName;
18        // alte lucruri
19    }
20 }
```

Linia 5: metoda main() este plasată pe stivă

Linia 6: variabila referință d este creată pe stivă (dar încă nu este nici un obiect de tip Dog)

Linia 7: un obiect de tipul Dog este creat pe în memoria heap; variabila d este o referință către acest obiect

Linia 9: o copie a variabilei referință d este transmisă ca argument la apelul metodei go()

Linia 10: metoda go() este plasată pe stivă cu parametrul dog ca variabilă locală

Linia 12: un nou obiect Collar este creat în memoria heap

Linia 15 setName() este adăugată pe stivă cu parametrul dogName ca variabilă locală

Exemplu concret de cum se alocă variabilele dintr-un program Java:

```
1 class Collar { }
2 class Dog {
3     Collar c; // variabila instanta
4     String name; // variabila instanta
5     public static void main(String[] arg){
6         Dog d; //variabila locala
7         d = new Dog();
8         d.go(d);
9     }
10    void go(Dog dog) {
11        // variabila locala: dog
12        c = new Collar();
13        dog.setName("Aiko");
14    }
15    void setName(String dogName) {
16        // variabila locala: dogName
17        name = dogName;
18        // alte lucruri
19    }
20 }
```

Linia 17: variabila instanță name referă acum către obiectul dogName

După terminarea liniei 19, setName() este finalizată și este scoasă de pe stivă împreună cu toate variabilele sale locale (dogName). Doar obiectul String "Aiko" rămâne deoarece mai există o referință către el

După terminarea liniei 13, metoda go() este scoasă de pe stivă. Obiectul String "Aiko" nu mai este referit

După terminarea liniei 8, metoda main este scoasă de pe stivă. Obiectul d nu mai este referit. Execuția programului se termină

Constructori

Constructori

- Orice clasă (inclusiv cele abstracte) trebuie să aibă cel puțin un constructor
- Nu înseamnă că trebuie neapărat implementat unul
- În lipsa codului explicit, Java generează implicit un constructor, caz în care variabilele instanță vor fi inițializate cu valorile implicite

```
class Person{
    String nume;
    int varsta;
    public Person(String n, int v){
        nume = n;
        varsta = v;
    }
    public Person() {}
    public static void main() {
        Person p = new Person("Ion", 28);
        // alt cod ...
    }
}
```

Supraîncărcarea constructorilor

- Java furnizează automat un constructor la crearea unei clase
- Definirea mai multor constructori cu același nume, dar parametri diferiți
- Programatorii vor scrie constructori proprii (inclusiv constructori care primesc argumente: semenea argumente sunt folosite la inițializare atunci când valorile obiectelor pot diferi)
- Dacă o clasă este instanțiabilă, Java furnizează automat un constructor
- **La crearea unui constructor propriu, constructorul furnizat automat încetează să existe**
- Ca și la alte metode, constructorii pot fi supraîncărcați;
- Supraîncărcarea constructorilor oferă o cale de a crea obiecte cu sau fără argumente inițiale, după necesități.

Supraîncărcarea constructorilor

Exemplu

```
public class SmartPhone {
    String model;
    String brand;
    int price;
    int storageCapacity;
    public SmartPhone(){}
    public SmartPhone(String model) { this.model = model; }
    public SmartPhone(int price) { this.price=price; }

    public SmartPhone(String model, String brand, int price){
        this(model);
        //this(price); !!! NU SE POATE !!!!!
        this.brand = brand;
        this.price = price;
    }
    public SmartPhone(SmartPhone s){
        this.model = s.model;
        this.brand = s.brand;
        this.price = s.price;
        this.storageCapacity = s.storageCapacity;
    }
}
```

Referința **this**

- Dacă avem 2 obiecte de tip Dreptunghi a și b de același tip, cum putem apela o singura metoda, *calculeazaArie()* pentru ambele obiecte ?
- Exista o singura metoda *calculeazaArie()*, cum știe metoda ca se apeleaza pentru obiectul a sau pentru obiectul b ?
- Exista un argument “secret” care se transmite metodei *calculeazaArie()*, si acest argument este referinta catre obiectul care este manipulat !

```
class Dreptunghi {
    int w, h;
    public Dreptunghi(int width, int height){
        w = width;
        h = height;
    }
    int calculeazaArie() { return w*h; }
}

public class DemoDreptunghi {
    public static void main(String[] args) {
        Dreptunghi a = new Dreptunghi(10,2);
        b = new Dreptunghi(3, 4);
        a.calculeazaArie();
        b.calculeazaArie();
    }
}
```

Referința **this**

- Dacă în interiorul unei metode dorim să accesăm referința la obiectul curent (cel pentru care s-a apelat metoda) folosim cuvântul cheie **this**
- **this** se folosește doar pentru acele cazuri speciale când este nevoie să se folosească referința la obiectul pentru care a fost apelată metoda !
- Metodele statice (metodele la nivel de clasă) nu au o referință **this** deoarece nu au un obiect asociat

Exemple de utilizare a lui **this**:

- Pentru a diferenția între variabilele de instanță și parametrii metodei.
- Pentru a apela un alt constructor din interiorul unui constructor (constructor chaining).
- Pentru a transmite obiectul curent ca parametru către o altă metodă sau constructor.
- Pentru a returna obiectul curent dintr-o metodă, pentru a susține metoda chaining.

Apelul unui alt constructor cu `this()`

Situația: În Java, putem apela un alt constructor al aceleiași clase folosind `this(...)`. Apelul trebuie să fie **prima instrucțiune** din constructor.

```
class Dreptunghi {  
    double w, h;  
    Dreptunghi() {  
        this(1, 1); // apelul altui constructor  
        System.out.println("Constructor fara parametri");  
    }  
    Dreptunghi(double l, double L) {  
        w = l;  
        h = L;  
        System.out.println("Constructor cu parametri");  
    }  
}
```

Rezultatul execuției:

```
Constructor cu parametri  
Constructor fără parametri
```

Observație: `this()` evită duplicarea codului și asigură inițializarea consistentă.

Referinta this

Exemplu de utilizare (1)

```
class Dreptunghi {  
    int w, h;  
    public Dreptunghi(int w, int h){  
        this.w = w;  
        this.h = h;  
    }  
    int calculeazaArie() { return w*h; }  
}  
public class DemoDreptunghi {  
    public static void main(String[] args) {  
        Dreptunghi a = new Dreptunghi(10,2);  
        b = new Dreptunghi(3, 4);  
        a.calculeazaArie();  
        b.calculeazaArie();  
    }  
}
```

Referința **this**

Exemplu utilizare: simulare operații ContBancar

```
private String numarCont;
private String numeDetinator;
int sold;
public ContBancar(String numarCont, String numeDetinator, int sold) {
    //diferenta intre variabile instanta si parametrii metodei
    this.numarCont = numarCont;
    this.numeDetinator = numeDetinator;
    this.sold = sold;
}
public ContBancar(String numarCont, String numeDetinator){
    this(numarCont, numeDetinator, 0); //apel constructor
}
public ContBancar depune(double amount) {
    this.sold += amount; // this acceseaza variabilele instanta
    System.out.println(this.numeDetinator + " depoziteaza $" + amount + ". Sold curent: " + this.sold);
    return this; // Returneaza 'this' —> method chaining
}
```

Referința `this`

Exemplu utilizare: simulare operații ContBancar

```
public ContBancar extage(double amount) {
    if (this.sold >= amount) {
        this.sold -= amount;
        System.out.println(this.numDetinator + "extrage"+amount+".Sold curent:" + this.sold);
    } else { System.out.println("Fonduri insuficiente!");}
    return this; // return 'this' —> method chaining
}
//transfera bani intre conturi
public void transfer(ContBancar targetAccount, double amount) {
    if (this.sold >= amount) {
        this.extage(amount); // 'this' refera obiectul curent
        targetAccount.depune(amount); // apelam metoda a obiectului dat ca parametru
        System.out.println("Transfera "+amount+" de la "+this.numDetinator+" spre "+ targetAccount.numDetinator);
    } else { System.out.println("Fonduri insuficiente!"); }
}
public void afiseazaDateCont() {
    System.out.println("Numar Cont: " + this.numarCont);
    System.out.println("Detinator: " + this.numDetinator);
    System.out.println("Sold: " + this.sold);
    System.out.println("_____");
}
// this in metoda
public void setNumDetinator(String accountHolder) {
    this.numDetinator = accountHolder;
}
```

Referință **this**

Exemplu utilizare: simulare operații ContBancar

```
public class Main {  
    public static void main(String[] args) {  
        ContBancar cont1 = new ContBancar("12345", "John Doe", 500);  
        ContBancar cont2 = new ContBancar("67890", "Jane Smith");  
        cont1.afiseazaDateCont();  
        cont2.afiseazaDateCont();  
  
        // depune si extrage  
        cont1.depune(200).extage(100).afiseazaDateCont();  
  
        // transfer intre conturi  
        cont1.transfer(cont2, 300);  
  
        // Display both account details after the transfer  
        cont1.afiseazaDateCont();  
        cont2.afiseazaDateCont();  
    }  
}
```

Quizz: constructori

<https://forms.gle/rqxBuFjhXx8a4rYDA>



Metode: vizibilitate, supraincarcare

Metode: cum funcționează un apel

```
import javax.swing.*;

public class KmToMiles {
    private static double convertKmToMi(double kilometri) {
        return kilometri * 0.621; // sunt 0.621 mile intr-un kilometru.
    }

    public static void main(String[] args) {
        //... variabile locale
        String kmStr;    // String km inainte de conversia la double.
        double km;        // Numar kilometri.
        double mi;        // Numar mile.
        //... Intrare
        kmStr = JOptionPane.showInputDialog(null, "Introduceti kilometri.");
        km = Double.parseDouble(kmStr);
        //... Calcule
        mi = convertKmToMi(km);
        //... Output
        JOptionPane.showMessageDialog(null, km + " kilometri sunt " + mi + " mile.");
    }
}
```

Metode

Cum funcționează un apel

Considerați atribuirea `mi=convertKmToMi(km)`; Pașii pentru procesarea acestei instrucțiuni sunt:

- 1 Evaluatează argumentele de-la-stânga-la-dreapta
- 2 Depune un nou cadru de stivă (stack frame) pe stiva de apeluri. Spațiu pentru parametri și variabilele locale (parametrul kilometri doar, aici). Starea salvată a metodei apelante (include adresa de retur)
- 3 Inițializează parametrii. La evaluarea argumentelor, acestea sunt asiguate parametrilor locali din metoda apelată.
- 4 Execută metoda.
- 5 Revine din metodă. Memoria folosită pentru cadrul de stivă pentru metoda apelată este scoasă de pe stivă.

Transmiterea parametrilor

- În Java transmiterea parametrilor la apelul metodelor se face numai prin valoare. Modificarile aduse parametrilor în interiorul unei metode nu se păstrează la revenirea din metoda respectivă.
- Dacă unul dintre parametrii unei metode are drept tip o clasă, aceasta înseamnă că la apel metoda va primi referința unui obiect al clasei.
 - Ceea ce se transmite prin valoare este chiar referința, NU obiectul indicat de ea
 - Metoda nu va putea modifica referința respectivă, dar va putea modifica obiectul indicat de ea.

Transmiterea parametrilor

Exemplu

```
class MyObject {
    int value;
}
class TransmitereParametri{
    public static void mP(int x) {
        x = 20; // nu schimba valoarea originala
    }
    //modifica variabila instantia
    public static void mObj(MyObject obj) {
        obj.value = 50; // schimba valoarea
    }
    public static void mRef(MyObject obj) {
        obj = new MyObject(); // nu schimba
            referinta
        obj.value = 100;
    }
}
```

```
public static void main(String[] args) {
    int a = 10;
    mP(a);
    System.out.println("Dupa modificare: " + a); //
        10
    //Exemplu cu obiect
    MyObject obj1 = new MyObject();
    obj1.value = 30;
    mObj(obj1);
    //Afiseaza 50
    System.out.println("Valoarea obj" + obj1.value);
    mRef(obj1);
    //Afiseaza tot 50
    System.out.println("Valoarea obj" + obj1.value);
}
}
```

Modificatori de acces pentru metode

- **private**: nu poate fi folosită de toate clasele; metoda este vizibilă doar în cadrul clasei
- **default** (nu se specifică nimic): nu poate fi folosită de toate clasele; metoda este vizibilă doar în pachetul din care face parte clasa
- **protected**: nu poate fi folosită de toate clasele; metoda este vizibilă doar în pachetul din care face parte clasa și în subclasele acesteia chiar dacă sunt în alte pachete
- **public**: - cel mai frecvent folosit; metoda este vizibilă tuturor
- **static**: nu e nevoie de obiecte pentru a folosi acest fel de metode; Dacă declarația metodei conține modificatorul static, este vorba de o metodă la nivelul clasei Metodele la nivelul clasei pot accesa doar constantele și variabilele clasei.

Supraîncărcarea unei metode

Supraîncărcarea unei metode Java înseamnă scrierea mai multor metode cu același nume dar cu argumente diferite

```
public class Dreptunghi {  
    private double w;  
    private double h;  
    public Dreptunghi() {  
        this.w = 1;  
        this.h = 1;  
    }  
    public Dreptunghi(double lat) {  
        this.w = lat;  
        this.h = lat;  
    }  
    public Dreptunghi(double w, double h)  
    {  
        this.w = w;  
        this.h = h;  
    }  
}
```

```
    public void redimensioneaza(double factor) {  
        this.w *= factor;  
        this.h *= factor;  
    }  
    public void redimensioneaza(double fw, double fh) {  
        this.w *= fw;  
        this.h *= fh;  
    }  
    public static void main(String[] args) {  
        Dreptunghi d1 = new Dreptunghi();  
        Dreptunghi d2 = new Dreptunghi(4);  
        Dreptunghi d3 = new Dreptunghi(5, 3);  
        d3.redimensioneaza(2);  
        d2.redimensioneaza(1.5, 0.5);  
    }
```

Ambiguitate la supraîncărcare

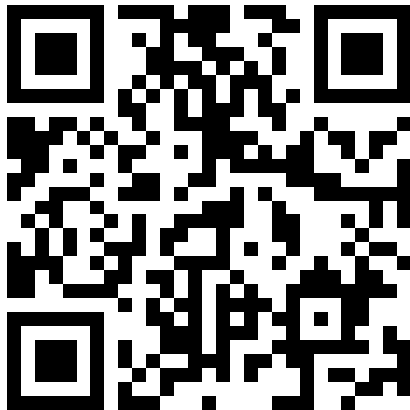
- La supraîncărcarea unei metode există riscul ambiguității
- O situație ambiguă este când compilatorul nu poate determina ce metodă să folosească
- Exemplu:

```
public void redimensioneaza(double fw, double fh) {  
    this.w *= fw;  
    this.h *= fh;  
}  
public int redimensioneaza(double fw, double fh) {  
    this.w *= fw;  
    this.h *= fh;  
}
```

- O supraîncărcare validă necesită cel puțin un argument de tip diferit sau un număr diferit de argumente

Quizz: modificatori de acces și supraîncărcarea metodelor

<https://forms.gle/JnDzDPZewmom25bY6>



Variabile și metode statice

Variabile la nivel de Clasa

- Variabile la nivel de clasă: variabile care sunt partajate de fiecare instanță a unei clase
- Numele instituției = "T.U. Cluj-Napoca"
- Fiecare angajat al unei instituții lucrează pentru aceeași instituție

```
private static String COMPANY_ID = "T.U. Cluj-Napoca";
```

Adăugați la clasa Dreptunghi o variabilă care să rețină numărul de dreptunghiuri create ! (discutie si exemplu daca variabila este sau nu este statica !

```
private static int totalDreptunghiuri = 0;  
...  
public Dreptunghi(int w, int h) {  
    this.w = w;  
    this.h = h;  
    totalDreptunghiuri ++;  
}
```

Variabile statice

Observații

- În JAVA se poate folosi **this** pentru a accesa o variabila statică dar NU este recomandat !
- Trebuie să înțelegem diferența dintre o variabila statica vs variabile instanță!

```
private static int totalDreptunghiuri = 0;
...
public Dreptunghi(int w, int h) {
    this.w = w;
    this.h = h;
    this.totalDreptunghiuri ++; // NUUUU
    Dreptunghi.totalDreptunghiuri ++; // ASA DA !!!
}
```

- Accesarea cu `numeClasa.numeVariabilaStatice` asigura claritatea codului și a semnificației variabilelor statice!

Variabile statice

O variabilă statică:

- Poate fi referită fie de clasă fie de obiect
- Instanțierea unui nou obiect de același tip nu sporește numărul de variabile statice
- Au aceeași valoare pentru toate instanțele clasei
- Este inițializată înaintea oricărui obiect de tipul clasei respective.



Metode Statice

- În Java se pot declara metode și variabile care să aparțină clasei, nu obiectului. Aceasta se face prin declararea lor ca static
- **Metodele statice:** declarate inserând cuvântul cheie static imediat după specificatorul de vizibilitate
- Aplicațiile Java de sine stătătoare, trebuie să-și inițieze execuția dintr-o metodă statică numită întotdeauna main și care are un singur tablou de String-uri ca parametru
- Despre metodele și variabilele statice:
 - Ce pot accesa ?
 - Cum se folosesc ?
 - Legarea
 - Suprascrierea
 - Alocarea memoriei

Metode Statice

Definiție și proprietăți

- **Metodă statică** este o metodă care aparține clasei
- Metodele statice pot fi apelate fără a crea o instanță a clasei.

```
MyClass.myStaticMethod();
```

- Sunt folosite pentru operațiuni care nu depind de variabile sau metode de instanță.

Proprietăți:

- 1 Metodele statice aparțin clasei
- 2 Nu pot accesa variabile sau metode instanță
- 3 Sunt folosite pentru funcții utilitare (helper functions) care nu necesită manipularea de date specifice instanței: `Math.pow()` sau `Collections.sort()`
- 4 Sunt apelate folosind numele clasei: `Math.sqrt(16);`

Metode statice

Exemplu

```
class Calculator {  
    public static int add(int a, int b) { return a + b; }  
    public static int multiply(int a, int b) { return a * b;}  
}  
  
public class Main {  
    public static void main(String[] args) {  
        //Apel de metode statice  
        int sum = Calculator.add(10, 5);  
        int product = Calculator.multiply(10, 5);  
        System.out.println("Sum: " + sum);  
        System.out.println("Product: " + product);  
    }  
}
```

Metode statice

Nu pot accesa variabile sau metode instanță

```
public class Calculator {  
    String modOperare; //basic, stiintific  
    public static int add(int a, int b) {  
        //Non-static field 'modOperare' cannot be referenced from a static context  
        System.out.println("ModOperare" + modOperare);  
        //'Calculator.this' cannot be referenced from a static context  
        System.out.println("ModOperare" + this.modOperare);  
        return a + b;  
    }  
    Calculator(String tip){this.modOperare = tip;}  
    String getModOperare(){return modOperare;}  
}
```

Metode statice

Se pot accesa variabile sau metode instanță prin intermediul obiectelor

```
public class Calculator {
    String modOperare; //basic, stiintific
    public static int add(int a, int b, Calculator c) {
        System.out.println("ModOperare" + c.getModOperare());
        return a + b;
    }
    public static int multiply(int a, int b) { return a * b;}
    Calculator(String tip){this.modOperare = tip;}
    String getModOperare(){return modOperare;}
}
class Main {
    public static void main(String[] args) {
        Calculator c = new Calculator("Basic");
        int sum = Calculator.add(10, 5, c);
        int product = Calculator.multiply(10, 5);
        System.out.println("Sum: " + sum);
        System.out.println("Product: " + product);
    }
}
```

Suprascrierea metodelor statice

La cursul de moștenire vom relua discuția despre metodele statice !
Ele nu pot fi suprascrise !

Tablouri

Definiție și Caracteristici

Definiție: Un șir este o structură de date care stochează o colecție de elemente de același tip, în ordine, într-o zonă de memorie continuă.

Caracteristici:

- Toate elementele dintr-un array au același tip de date.
- Dimensiunea unui array este fixă și trebuie specificată la momentul inițializării.
- Indexarea începe de la 0.

Tablouri unidimensionale (1D)

Declarație și Inițializare:

```
int[] myArray = new int[5]; // Sir de int cu 5 elemente  
myArray[0] = 10; // Initializarea primului element
```

Inițializare directă:

```
int[] myArray = {10, 20, 30, 40, 50}; // Declaratie si initializare
```

Accesarea elementelor:

```
System.out.println(myArray[2]); // Afiseaza valoarea 30
```

Parcurgere folosind for clasic:

```
int[] myArray = {1, 2, 3, 4, 5};  
for (int i = 0; i < myArray.length; i++) {  
    System.out.println(myArray[i]);  
}
```

Parcurgere folosind for-each:

```
for (int num : myArray) { System.out.println(num); }
```

Obținerea dimensiunii unui sir:

```
System.out.println(myArray.length); // Afiseaza 5
```

Tablouri bidimensionale (2D)

Declarație și Inițializare:

```
int[][] matrix = new int[3][3]; // Matrice 3x3
matrix[0][0] = 1; // Initializarea unui element
```

Inițializare directă:

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
```

Parcurgere:

```
int[][] matrix = {
    {1, 2, 3},
    {4, 5, 6},
    {7, 8, 9}
};
for (int i = 0; i < matrix.length; i++) {
    for (int j = 0; j < matrix[i].length; j++) {
        System.out.print(matrix[i][j] + " ");
    }
    System.out.println();
}
```

Șiruri de obiecte

Definiție: Un șir de obiecte este un array care conține instanțe ale unei clase, în loc de tipuri de date primitive.

Caracteristici:

- Toate elementele dintr-un șir de obiecte sunt instanțe ale aceleiași clase.
- Array-ul poate conține și valori `null` (fără referință).
- Șirurile de obiecte pot fi create pentru orice tip de obiecte definite de utilizator.

Declarație, Inițializare, Accesare

Declarație și alocare de memorie:

```
class Car {  
    String model;  
    Car(String model) { this.model = model; }  
}  
Car[] cars = new Car[3]; // Alocare de memorie pentru 3 obiecte Car
```

Inițializare elemente individuale:

```
cars[0] = new Car("Toyota");  
cars[1] = new Car("Honda");  
cars[2] = new Car("BMW");
```

Declarație și inițializare directă:

```
Car[] cars = {  
    new Car("Toyota"),  
    new Car("Honda"),  
    new Car("BMW")  
};
```

Accesarea elementelor:

```
System.out.println(cars[0].model); // Afiseaza "Toyota"
```

Parcurgere, tratarea valorilor null

Folosind un for clasic:

```
for (int i = 0; i < cars.length; i++) {  
    System.out.println(cars[i].model);  
}
```

Folosind un for-each:

```
for (Car car : cars) {  
    System.out.println(car.model);  
}
```

Valoarea null într-un sir de obiecte:

```
Car[] cars = new Car[3]; // Alocat, dar neinitializat  
cars[0] = new Car("Toyota");  
cars[1] = null; // Referinta nula  
cars[2] = new Car("BMW");  
if (cars[1] != null) {  
    System.out.println(cars[1].model); // Se evita NullPointerException  
} else { System.out.println("Obiect nul."); }
```

Modificarea Obiectelor dintr-un Sir

Accesarea și modificarea obiectelor dintr-un sir:

```
Car[] cars = { new Car("Toyota"), new Car("Honda"), new Car("BMW") };

// Modificarea proprietatilor unui obiect
cars[0].model = "Tesla";

System.out.println(cars[0].model); // Afiseaza "Tesla"
```

Modificarea unui element din sir:

```
cars[1] = new Car("Ford"); // Inlocuieste obiectul la indexul 1
System.out.println(cars[1].model); // Afiseaza "Ford"
```

Siruri de Obiecte ca Parametri

- Când se transmite un array ca parametru, Java transmite o copie a referinței acelui array, nu o copie a valorilor array-ului în sine.
- Această referință indică același obiect de tip array în memorie, așa că modificările asupra elementelor array-ului în metoda apelată vor afecta array-ul original.

Exemplu:

```
public void printCarModels(Car[] cars) {  
    for (Car car : cars) {  
        System.out.println(car.model);  
    }  
}  
  
// Apelarea metodei  
Car[] cars = { new Car("Toyota"), new Car("Honda") };  
printCarModels(cars);
```

Crearea unei Copii a Șirului

Siruri de numere primitive

```
class Example {  
    public static void modifyArray(int[] array) {  
        array[0] = 99; // Modifica primul element  
    }  
    public static void main(String[] args) {  
        int[] numbers = {1, 2, 3};  
        System.out.println("Inainte de modificare: " + numbers[0]);  
        // Crearea unei copii a sirului  
        int[] copyOfNumbers = numbers.clone();  
        modifyArray(copyOfNumbers);  
        System.out.println("Dupa modific.(sir original):"+numbers[0]);  
        System.out.println("Dupa modific.(copie):"+ copyOfNumbers[0]);  
    }  
}
```

Explicație:

- 'numbers.clone()' creează o copie a array-ului original.
- Modificările făcute asupra 'copyOfNumbers' nu afectează 'numbers'.
- Sirul original rămâne neschimbat, în timp ce copia este modificată.

Crearea unei Copii a Șirului

Siruri de obiecte

```
class Example {  
    public static void modifyCarArray(Car[] array) {  
        Car[0] = new Car("Tesla") // Modifica primul element  
    }  
    public static void main(String[] args) {  
        Car[] cars = {new Car("Toyota"), new Car("Honda"), new Car("BMW")};  
        System.out.println("Inainte de modificare: " + cars[0]);  
        // Crearea unei copii a sirului  
        Car[] copyOfCars = cars.clone();  
        modifyCarArray(copyOfCars);  
        System.out.println("Dupa modific.(sir original): "+cars[0]);  
        System.out.println("Dupa modific.(copie): "+ copyOfCars[0]); //sunt la fel !!!  
    }  
}
```

Explicație:

- 'cars.clone()' creează o copie a referințelor elementelor !
- Dacă se modifică un obiect referit in copyOfCars el se va modifica si in cars !

Tipuri de copiere a șirurilor în JAVA

- **Copie Superficială (Shallow Copy):**

- Metodele 'clone()' sau 'System.arraycopy()' creează o copie a referințelor elementelor.
- Dacă șirul conține obiecte, copiile vor conține aceleași referințe.

- **Copie Profundă (Deep Copy):**

- Necesită o implementare manuală pentru a crea copii independente ale obiectelor.
- **Exemplu:**

```
Car[] originalArray = {new Car("Toyota"), new Car("Honda")};  
  
Car[] deepCopyArray = new Car[originalArray.length];  
for (int i = 0; i < originalArray.length; i++) {  
    deepCopyArray[i] = new Car(originalArray[i].getModel());  
}
```

Returnarea unui Șir de Obiecte

Exemplu:

```
public Car[] createCarArray() {  
    Car[] cars = {  
        new Car("Toyota"),  
        new Car("Honda")  
    };  
    return cars;  
}  
  
// Utilizarea sirului returnat  
Car[] cars = createCarArray();  
System.out.println(cars[0].model); // Afiseaza "Toyota"
```

Concluzii siruri de obiecte

- Sirurile de obiecte sunt un mod eficient de a gestiona mai multe instanțe ale unei clase.
- Este important să tratați valorile `null` pentru a evita erorile de tip `NullPointerException`.
- Array-urile de obiecte pot fi trimise ca parametri, returnate din metode și modificate individual.

Reguli proiectare clase

Inițializați datele; Creați clase pentru a cuprinde tipuri multiple !

- Inițializați întotdeauna datele
 - Java nu va inițializa variabilele locale, dar va inițializa variabilele instanță ale obiectelor
 - Nu vă bazați pe valorile implicite, ci inițializați variabilele explicit
- Nu folosiți prea multe tipuri într-o clasă:
 - Înlocuiți folosirile multiple înrudite ale tipurilor de bază cu alte clase. Spre exemplu:

```
private String strada;  
private String oras;  
private String stat;  
private String tara;  
private int codPostal;
```

```
class Adresa{  
    private String strada;  
    private String oras;  
    private String stat;  
    private String tara;  
    private int codPostal;  
}
```

Folosiți încapsularea

- Păstrați datele întotdeauna private
 - Schimbările în reprezentarea datelor nu vor afecta utilizatorii claselor; erorile sunt mai ușor de detectat
- Pentru accesul sau modificarea valorilor variabilelor instanță se vor implementa metode accesorii și mutatori (getter și setter)
- Nu toate câmpurile necesită accesorii și mutatori individuali
 - Exemplu: Angajat - obține și setează salariul, dar nu și data angajării (nu se schimbă o dată creată)

Contra-exemplu de cod pentru încapsulare:

```
class Foo {  
    public int left = 9;  
    public int right = 3;  
    public void setLeft(int leftNum) {  
        left = leftNum;  
        right = leftNum/3;  
    }  
}
```

- Variabila right va fi totdeauna 1/3 din left???
- Nu, deoarece utilizatorii clasei Foo pot seta direct valoarea variabilelor left și right, fără să apeleze metoda setLeft()

Folosiți o formă standard pentru definirea claselor

- Pentru fiecare secțiune, scrieți în ordine:
 - constante
 - constructori
 - metode
 - metode statice
 - variabile instanță
 - variabile statice
- De ce? Utilizatorii sunt mai interesați de interfața publică decât de datele private și mai mult de metode decât de date
- Faceți numele claselor și metodelor să reflecte responsabilitățile acestora
- O convenție bună este:
 - Numele clasei: substantiv (ex. Comanda) sau substantiv+adjectiv (ex. ComandaUrgenta)
 - Numele metodelor: verbe; accesorii încep cu "get"; mutatorii încep cu "set"

Divizați clasele cu prea multe responsabilități

```
class PachetCarti { // nu este recomandat
    public void PachetCarti() { ... }
    public void amesteca() { ... }
    public void obtineValoareaMaxima() { ... }
    public void obtineFelulMaxim() { ... }
    public void rangulMaxim() { ... }
    public void deseneaza() { ... }
    private int[] valoare;
    private int[] fel;
    private int carti;
}
```

Creați clasa Carte!!